

```

int divide(int a, int b) {
    //如果被除数是最小值
    if(a==INT_MIN){
        if(b==1)return INT_MIN;
        if(b==-1)return INT_MAX;
    }
    //如果除数是最小值
    if(b==INT_MIN){
        return a==INT_MIN?1:0;
    }
    //如果被除数是0
    if(a==0)return 0;
    //取反操作 rev最后判断符号
    bool rev = false;
    if(a>0){
        a=-a;
        rev=!rev;
    }
    if(b>0){
        b=-b;
        rev=!rev;
    }
    int ans=0;
    while(a<=b){
        //被除数 除数 除数每次翻一倍
        int temp=b;
        unsigned int count=1;
        while(temp>=0xc0000000&&a<=temp+temp){
            count+=count;
            temp+=temp;
        }
        ans+=count;
        a-=temp;
    }
    return rev?-ans:ans;
}

```

剑指 offer 第一题,求两数除法不让使用一些现有的运算符,最高 32 位存储,也就是说最大 int 存储,如果计算过程中溢出 直接返回 INT_MAX, 以上述条件开始解题,首先明确 32 位也就是 int 有符号 int 的范围 -2147483648 ~ 2147483647

所以看来 负数是比较正数多一位的,所以正数更容易溢出,所以把除数和被除数变为负号进行操作,这是前提条件,具体怎么实现,除法就是看被除数 能减去除数几次,利用类似二分查找的方法 ,用一个变量记录当前减去的次数,然后每次除数翻倍,除数与当前减去次数同步翻倍,直到最后被除数已经小于除数了,已经不能再减少了 这时候把当前次数更新到 ans 中,然后把被除数减去当前的除数,开始下一轮循环,直到最后被除数小于除数了退出循环 判断当前符号返回